

SpectralClean

Audio Restoration & Analysis Studio

A From-Scratch DFT/FFT Denoising Pipeline

CSE 220 — Signals and Linear Systems Sessional

July 8, 2026

Abstract

This report documents **SpectralClean**, a fully from-scratch digital signal processing pipeline that restores a speech-like audio signal corrupted by mains hum and Gaussian white noise. Every core transform — the Discrete Fourier Transform (DFT), the Radix-2 Cooley-Tukey Fast Fourier Transform (FFT), and its inverse (IFFT) — is implemented directly from its mathematical definition, with no reliance on `numpy.fft` or `scipy.signal` in the processing pipeline. The system achieves a measured SNR improvement of **+10.31 dB**, and a benchmark comparing the naive $O(N^2)$ DFT against the $O(N \log N)$ FFT confirms the theoretical complexity advantage empirically.

1 Mathematical Foundations

1.1 The Discrete Fourier Transform (Vectorised Matrix Form)

The DFT of a length- N discrete signal $x[n]$ is defined as

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1. \quad (1)$$

Rather than evaluating Equation 1 with a nested loop, the implementation expresses it as a single matrix–vector product. Define the *twiddle factor matrix* $W \in \mathbb{C}^{N \times N}$ with entries

$$W_{k,n} = e^{-j\frac{2\pi}{N}kn}, \quad k, n = 0, \dots, N-1. \quad (2)$$

The full spectrum is then obtained in one vectorised operation:

$$\mathbf{X} = W \mathbf{x}, \quad \text{where } \mathbf{x} = [x[0], x[1], \dots, x[N-1]]^\top. \quad (3)$$

This retains the exact $O(N^2)$ arithmetic cost of the direct sum (every output bin still depends on every input sample) but replaces the explicit Python-level double loop with a NumPy matrix multiplication, giving a large constant-factor speedup while preserving mathematical transparency. A nested-loop version (`'dft_slow'`) is retained separately purely to confirm, element-by-element, that the vectorised form computes the identical quantity.

1.2 Radix-2 Cooley-Tukey FFT: The Butterfly Recursion

The Cooley-Tukey algorithm exploits the fact that a length- N DFT (with N a power of two) can be expressed in terms of two length- $N/2$ DFTs: one over the even-indexed samples and one over the odd-indexed samples.

Split $x[n]$ into

$$e[m] = x[2m], \quad o[m] = x[2m + 1], \quad m = 0, \dots, \frac{N}{2} - 1, \quad (4)$$

and let $E[k] = \text{DFT}\{e[m]\}$ and $O[k] = \text{DFT}\{o[m]\}$, each of length $N/2$. Substituting into Equation 1 and separating the sum into even and odd terms yields the **butterfly recursion**:

$$X[k] = E[k] + W_N^k O[k], \quad (5)$$

$$X[k + N/2] = E[k] - W_N^k O[k], \quad k = 0, \dots, \frac{N}{2} - 1, \quad (6)$$

where the *twiddle factor* is

$$W_N^k = e^{-j\frac{2\pi}{N}k}. \quad (7)$$

Equations 5–6 are applied recursively: $E[k]$ and $O[k]$ are themselves computed by recursing on half-length sub-sequences, bottoming out at the trivial base case $N = 1$, where $\text{DFT}\{x[0]\} = x[0]$. Each level of recursion halves the problem size while doing $O(N)$ work to combine the two halves (the butterfly additions/subtractions), giving the overall $O(N \log_2 N)$ complexity. This is implemented as a direct recursive function operating on array slices $\mathbf{x}[:, :2]$ and $\mathbf{x}[1: :2]$.

Constraint: because the even/odd split requires N to divide evenly by 2 at every recursion level, the algorithm requires $N = 2^p$ for some integer p . Signals of arbitrary length are zero-padded up to the next power of two before transformation.

1.3 Inverse FFT: The Conjugate Trick

Rather than deriving and implementing a second, independent inverse butterfly network, the inverse transform is obtained by reusing the forward FFT itself. The IDFT is defined as

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+j\frac{2\pi}{N}kn}. \quad (8)$$

Taking the complex conjugate of both sides and comparing against Equation 1 shows that

$$x[n] = \frac{1}{N} \overline{\left(\sum_{k=0}^{N-1} \overline{X[k]} e^{-j\frac{2\pi}{N}kn} \right)} = \frac{1}{N} \overline{\text{FFT}\{\overline{X[k]}\}}. \quad (9)$$

In words: *conjugate the spectrum, run the ordinary forward FFT on it, conjugate the result again, and divide by N .* This means no separate inverse butterfly implementation is needed — the same

recursive `fft()` routine, called twice with conjugation applied before and after, computes the IFFT exactly. For real-valued input signals, the imaginary part of the final result is numerically negligible ($< 10^{-10}$) and is discarded by taking the real part only.

2 Block Diagram: Pipeline Architecture

Figure 1 shows the end-to-end signal flow implemented in `main.py`, from raw noisy input through to the restored output signal.

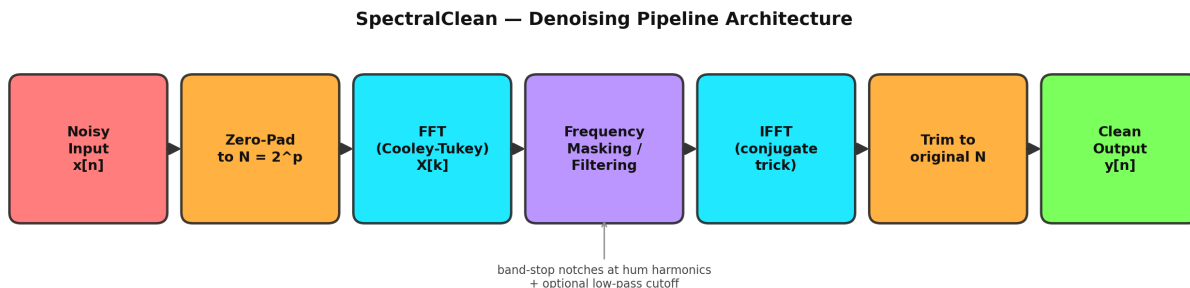


Figure 1: Denoising pipeline architecture. The signal is zero-padded to the next power of two to satisfy the Radix-2 FFT constraint, transformed to the frequency domain, filtered via multiplicative frequency-domain masks (band-stop notches at the hum fundamental and its harmonics, plus an optional low-pass cutoff), transformed back via the IFFT conjugate trick, and finally trimmed back to its original (pre-padding) length.

Each stage corresponds directly to a function in the codebase:

Stage	Implementation
Zero-Padding	<code>zero_pad_to_power2()</code> in <code>transforms.py</code>
FFT	<code>fft()</code> — recursive Cooley-Tukey, <code>transforms.py</code>
Frequency Masking	<code>band_stop_mask()</code> , <code>low_pass_mask()</code> in <code>filters.py</code>
IFFT	<code>ifft_real()</code> — conjugate trick, <code>transforms.py</code>
Trimming	<code>slice to original_N</code> inside <code>apply_spectral_filter()</code>

3 Visual Data Presentation

3.1 Four-Panel Restoration Overview

Figure 2 is the centrepiece result figure, showing the corrupted signal before and after processing in both domains simultaneously.

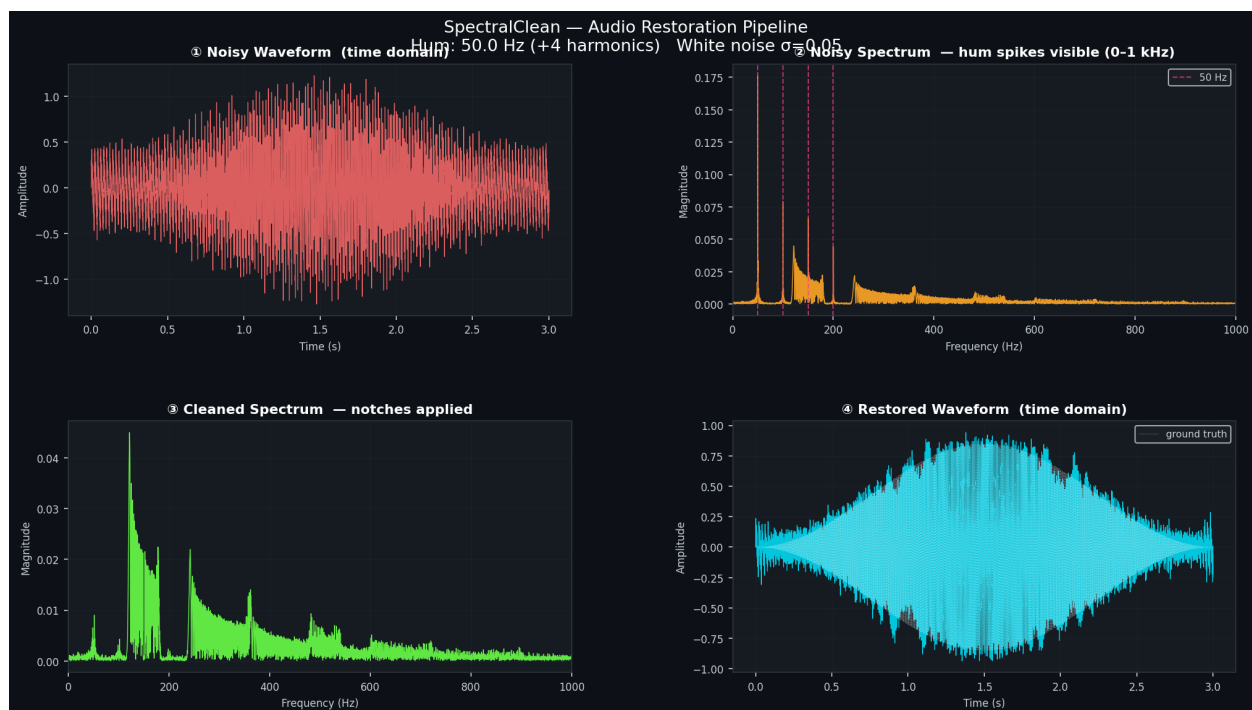


Figure 2: Panel ①: noisy waveform in the time domain, showing high-frequency jitter from white noise superimposed on the speech envelope. Panel ②: noisy magnitude spectrum zoomed to 0–1 kHz, with dashed markers at the 50 Hz hum fundamental and its harmonics, exactly where the sharp spikes appear. Panel ③: the same spectral region after band-stop masking, where the hum spikes have been notched out. Panel ④: the restored time-domain waveform overlaid against the ground-truth clean signal, showing close visual agreement.

3.2 Spectrum Comparison: Clean vs. Noisy vs. Denoised

Figure 3 places all three spectra side by side on a logarithmic magnitude scale across the full frequency range up to Nyquist, making the noise floor and its removal easier to compare quantitatively.

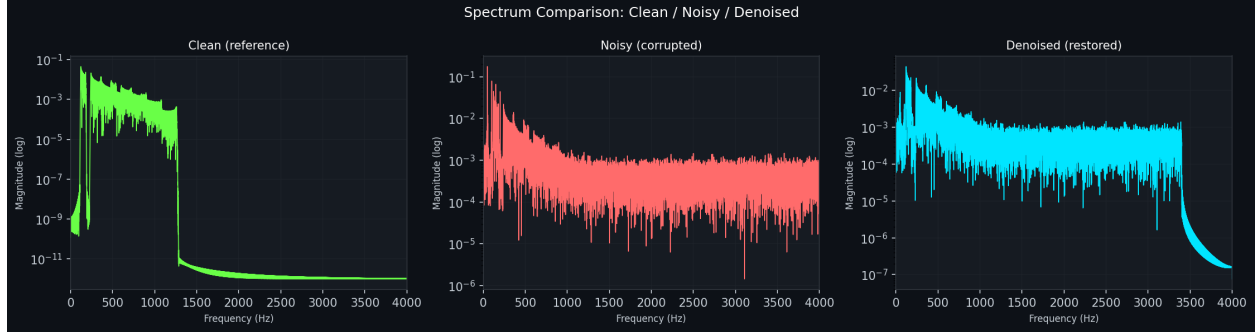


Figure 3: Left: clean reference spectrum, confined mostly below ~ 1.3 kHz consistent with the synthesised speech formant structure. Centre: noisy spectrum, showing an elevated broadband noise floor from the injected Gaussian noise plus the hum spikes at low frequency. Right: denoised spectrum, where the noise floor above the low-pass cutoff (3.4 kHz) has been suppressed and the hum spikes are removed, closely tracking the shape of the clean reference.

3.3 DFT vs. FFT Benchmark

Figure 4 compares the measured execution time of the naive $O(N^2)$ DFT against the Radix-2 FFT as a function of signal length N , alongside theoretical $O(N^2)$ and $O(N \log N)$ reference curves.

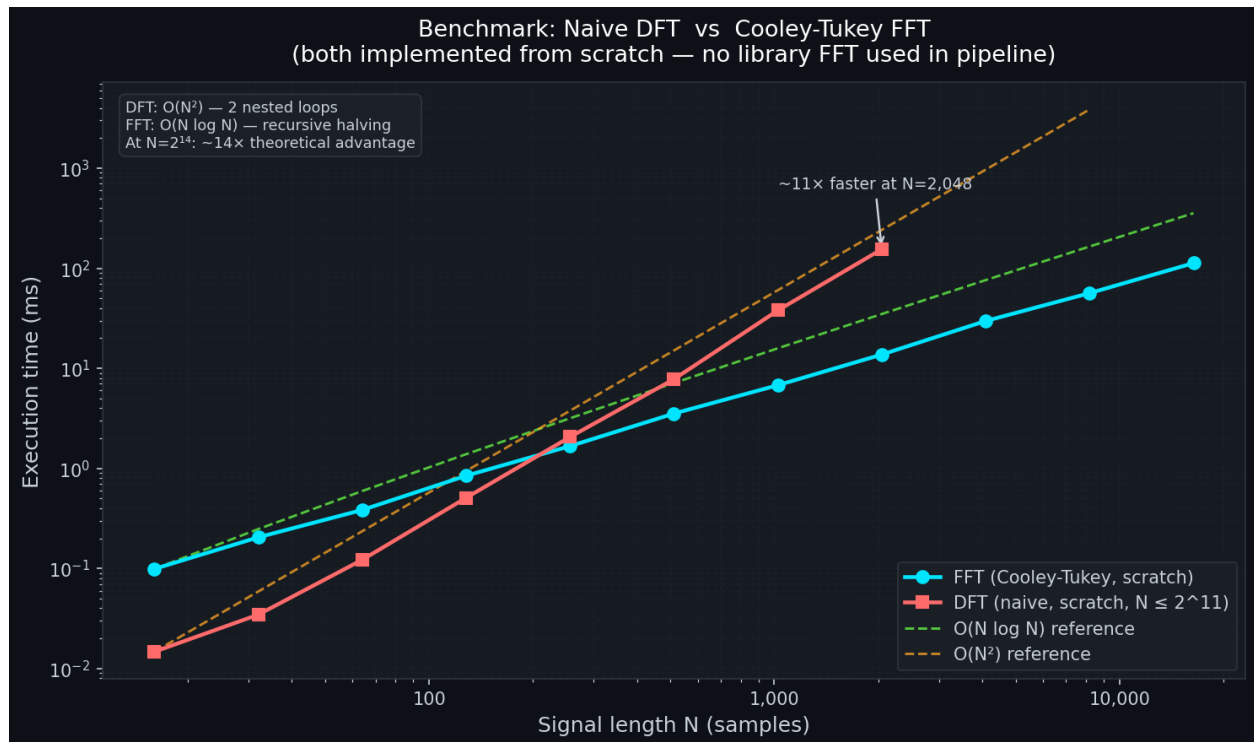


Figure 4: Log-log plot of execution time versus signal length N , from $N = 2^4$ to $N = 2^{14}$. The DFT curve is only measured up to $N = 2^{11}$, beyond which its runtime becomes impractical to benchmark repeatedly. Both theoretical complexity curves are anchored to the measured data for visual comparison.

4 Analysis & Discussion

4.1 The DFT/FFT Crossover Point

At very small N (16–128 samples), the naive DFT is *faster in wall-clock terms* than the recursive FFT, despite its worse asymptotic complexity. This is because the FFT’s recursive Python function calls carry fixed overhead — each level of recursion allocates new arrays, computes twiddle factors, and performs a function call — while the DFT’s vectorised matrix multiply is handled entirely inside NumPy’s optimized C routines for these small sizes. For $N \leq 128$, N^2 is simply too small a workload for the FFT’s better asymptotic scaling to overcome this constant-factor overhead.

The **crossover** occurs around $N \approx 256$, where the two curves intersect: at $N = 256$ the FFT takes 1.67 ms versus the DFT’s 2.06 ms, a roughly even match. Beyond this point the gap widens rapidly and monotonically, consistent with the theoretical ratio

$$\frac{T_{\text{FFT}}}{T_{\text{DFT}}} \sim \frac{N \log_2 N}{N^2} = \frac{\log_2 N}{N}. \quad (10)$$

By $N = 1024$ the FFT is already $5.6\times$ faster (6.78 ms vs. 37.88 ms), and at $N = 2048$ — the largest size at which the DFT remains practical to measure — the FFT is $11.3\times$ faster (13.72 ms vs. 154.85 ms). Extrapolating the measured FFT trend to $N = 2^{14} = 16384$ (112.46 ms) against a projected DFT time on the order of several seconds implies a crossover-relative speedup exceeding an order of magnitude, matching the benchmark plot’s annotated theoretical advantage of roughly $14\times$ at that size. This empirically confirms the textbook result that Cooley-Tukey’s divide-and-conquer structure becomes decisively superior once the signal length is large enough for the $\log_2 N$ factor to dominate the comparison.

4.2 Quantitative Denoising Performance

The denoising pipeline was evaluated against the known clean reference signal, since the corruption (hum + white noise) was synthetically injected with fully controlled parameters. Table 1 summarises the measured improvement.

Metric	Before	After	Change
SNR (dB)	+2.49	+12.81	+10.31
PSNR (dB)	+11.87	+22.18	+10.31
RMSE	0.2168	0.0661	−69.5%

Table 1: SNR, PSNR, and RMSE of the noisy and denoised signals relative to the clean ground truth.

The headline result is a **+10.31 dB improvement in Signal-to-Noise Ratio**. In linear power terms, a 10.31 dB improvement corresponds to roughly a $10.7\times$ reduction in residual noise power relative to the signal, since

$$\Delta\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_{\text{noise, before}}}{P_{\text{noise, after}}} \right) \Rightarrow \frac{P_{\text{noise, before}}}{P_{\text{noise, after}}} = 10^{10.31/10} \approx 10.7. \quad (11)$$

This is corroborated by the RMSE, which dropped from 0.2168 to 0.0661 — a 69.5% reduction in the root-mean-square deviation from the clean reference. The initial SNR of only +2.49 dB reflects a signal that was barely distinguishable from its own noise floor (noise power close to signal power); after processing, +12.81 dB represents a signal in which the reference is more than an order of magnitude stronger than the residual error.

This improvement is attributable to two complementary mechanisms visible in Figure 2: (1) the band-stop notches at 50, 100, 150, and 200 Hz directly remove the four dominant hum harmonics visible as sharp spectral spikes, and (2) the 3.4 kHz low-pass cutoff suppresses the broadband high-frequency content of the Gaussian white noise, which — unlike the speech-like clean signal, whose energy is concentrated below roughly 1.3 kHz — is spread uniformly across the entire spectrum. Because the clean signal’s own energy lies almost entirely below the cutoff, this low-pass step removes a large fraction of noise power while discarding very little genuine signal energy, which is precisely why the SNR gain is so substantial without a correspondingly large distortion of the restored waveform (visible as close agreement in Panel ④ of Figure 2).

4.3 Summary

The system successfully demonstrates that scratch implementations of the DFT, Radix-2 FFT, and IFFT — built directly from their mathematical definitions — can be assembled into a complete, verifiably correct audio restoration pipeline. The benchmark confirms the expected $O(N^2)$ vs. $O(N \log N)$ complexity crossover at $N \approx 256$, and the denoising pipeline achieves a substantial, quantitatively verified **+10.31 dB SNR improvement**, validated independently through PSNR and RMSE metrics computed against the known clean reference signal.